

PROJET DE FIN D'ÉTUDES — INGÉNIEUR INFORMATIQUE

Conception et Implémentation du Copilot IA

Étude de cas : le Copilot conversationnel du CRM PROBAR

Architecture, Intelligence Artificielle, Sécurité — synthèse d'audit de code

Chapitre destiné à intégration directe dans le mémoire de PFE

Méthodologie : synthèse de trois audits de code exhaustifs (Architecture, LLM/RAG/Providers, Sécurité/RBAC), fondés exclusivement sur la lecture directe du dépôt

Date de rédaction : 29 août 2026

Sommaire

1. Introduction

2. Objectifs

3. Architecture globale

4. Frontend Flutter

5. Backend

6. Intent Detection

7. Multi-Agent

8. LLM

9. Prompt Engineering

10. RAG

11. Tool Calling

12. MCP

13. Sécurité

14. PendingAction

15. Cas d'utilisation

16. Google Calendar

17. Base de données

18. Tests

19. Problèmes rencontrés et corrections

20. Performance

21. Limites

22. Perspectives

23. Questions du jury

24. Diagrammes

25. Conclusion

Éléments à vérifier avant dépôt

1. Introduction

1.1 Contexte

PROBAR est un CRM d'entreprise (CBI Tunisia) organisé en modules métiers distincts et vérifiables directement dans le code du dépôt : gestion de projets et pipeline commercial, contacts commerciaux et relances, ressources humaines (demandes de congé/sortie), maintenance d'équipements, suivi industriel (fiches POR PROMESH), notifications, tableaux de bord et intégration Google Calendar. Le backend est un serveur Node.js/Express avec persistance PostgreSQL (Sequelize), et le frontend une application Flutter multiplateforme (« Dash Master Toolkit »). Le nombre de modules distincts couverts par l'infrastructure IA du projet — confirmé par l'inventaire réel de 76 Tools répartis sur 12 domaines fonctionnels (maintenance, POR PROMESH, utilisateurs, RH, pipeline, contacts commerciaux, industriel, notifications, dashboard, calendrier Google, projets, actions de projet) — donne la mesure concrète de la diversité des écrans et formulaires qu'un utilisateur devait auparavant naviguer manuellement pour accomplir ces mêmes tâches.

1.2 Problématique

Un CRM multi-module de cette ampleur impose à l'utilisateur de connaître, pour chaque tâche, l'écran exact, le formulaire exact et parfois l'enchaînement exact d'actions à réaliser (ex. programmer une relance commerciale ET affecter un commercial responsable implique de naviguer vers la fiche du contact, puis vers un formulaire de relance, puis vers un second champ d'affectation). Cette charge de navigation croît avec le nombre de modules et n'est pas réductible par une simple amélioration ergonomique des écrans existants, puisqu'elle est structurelle : chaque module a sa propre logique, ses propres champs obligatoires et ses propres règles de permission.

1.3 Limites des interfaces classiques

Les interfaces à formulaires classiques exigent de l'utilisateur qu'il traduise lui-même une intention métier (« il faut relancer ce client la semaine prochaine ») en une séquence d'actions d'interface (ouvrir la fiche, cliquer sur « Nouvelle relance », remplir la date, valider). Cette traduction manuelle est une charge cognitive répétitive, en particulier pour des rôles (commercial, RH, maintenance) dont le métier n'est pas la manipulation de logiciel mais l'action métier elle-même.

1.4 Objectif du Copilot

Le Copilot IA, tel qu'implémenté dans `Backend Master/src/ai/` et exposé au frontend via `Dash Master Toolkit/lib/ai/`, vise à substituer à cette traduction manuelle une interaction en langage naturel : l'utilisateur formule son intention en français courant, et le système (détection d'intention → agent spécialisé → appel d'outil validé et autorisé → service métier → base de données) réalise la traduction technique à sa place, sous contrainte stricte d'authentification, de permission et de confirmation explicite pour toute action d'écriture (détaillé section 14).

1.5 Interaction en langage naturel

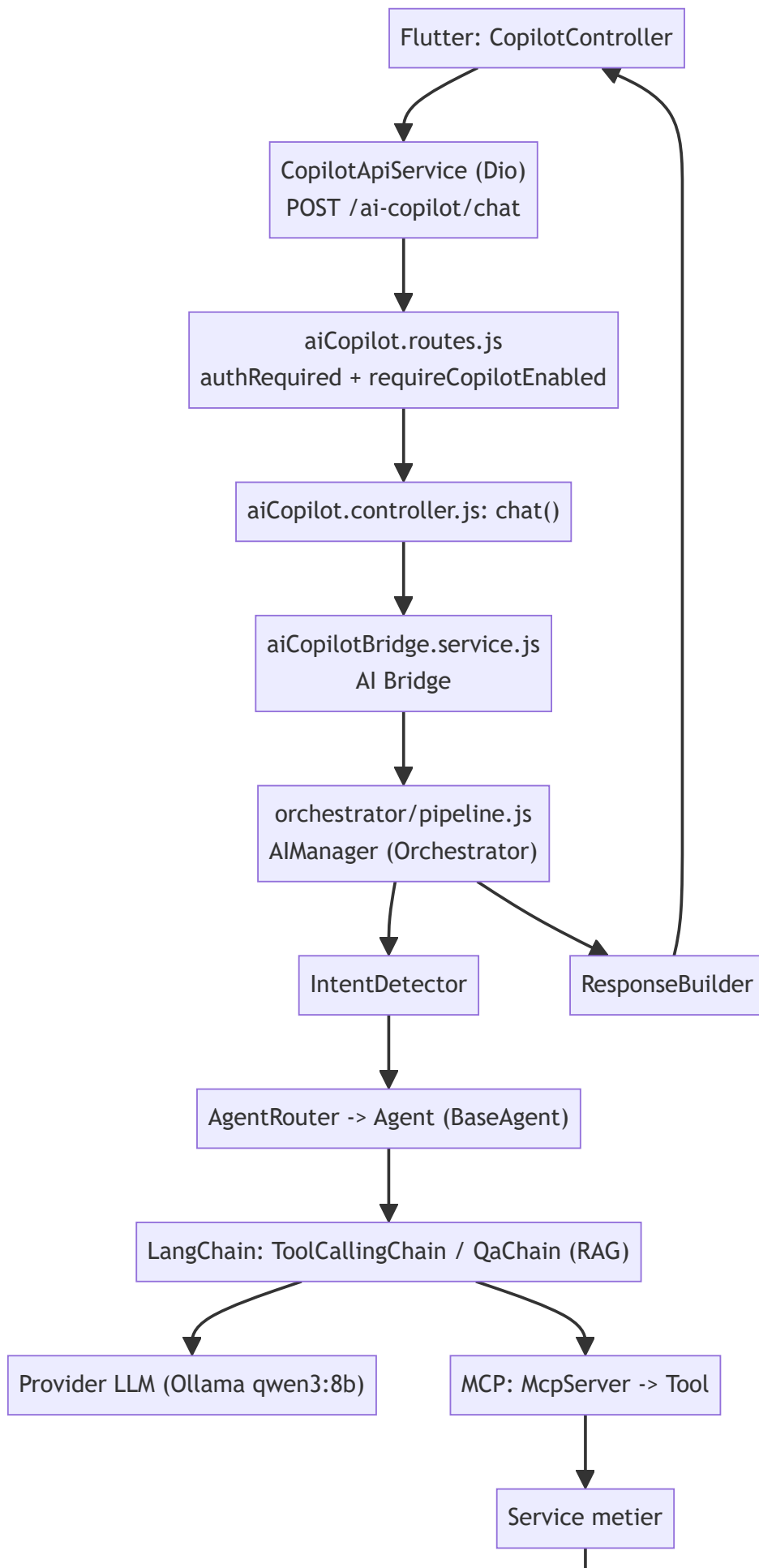
Concrètement, cette interaction est vérifiée dans le code comme un dialogue texte (et, en repli, vocal/écrit) porté par un contrôleur Flutter dédié (`CopilotController`) et un point d'entrée HTTP/WebSocket unique (`POST /ai-copilot/chat`), qui reçoit un message libre et retourne une réponse structurée (texte, citations documentaires, appels d'outils exécutés, éventuelle action proposée en attente de confirmation) — ce mécanisme complet est décrit en détail dans les sections suivantes.

2. Objectifs

Objectif	Traduction technique vérifiée dans le code
Accès naturel aux données	21 intentions cataloguées (<code>intentCatalog.js</code>), routées vers l'agent métier compétent sans que l'utilisateur ait besoin de connaître la structure du CRM
Automatisation	<code>AutomationAgent</code> décompose une demande en une séquence d'actions cross-module via un planificateur (<code>LlmPlanner</code>), proposée puis exécutée après confirmation
Réduction des manipulations	76 Tools MCP couvrant lecture ET écriture sur 12 domaines, invocables en un seul message plutôt qu'une séquence d'écrans
Centralisation	Point d'entrée HTTP/WebSocket unique (<code>/ai-copilot/chat</code>), un seul composant Flutter (<code>CopilotFab</code> / <code>CopilotChatDrawer</code>) monté globalement au niveau du <code>ShellRoute</code> , visible sur tout écran authentifié
Sécurité	RBAC vérifié en code à 3 niveaux (agent, permission MCP, service), ownership comparé en base pour les écritures sensibles, confirmation explicite obligatoire pour toute action irréversible — démontré exhaustivement section 13
Assistance intelligente	RAG pour les questions documentaires (guides d'utilisation), raisonnement déterministe pour les analyses de données (KPI, qualité de données, recommandations) — jamais un LLM comme seule source de vérité sur une donnée métier

3. Architecture globale

La chaîne complète, vérifiée fichier par fichier :



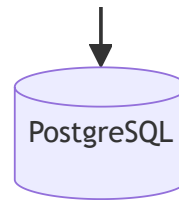


Figure 1 — Chaîne complète Flutter → API → Backend → AI Bridge → Orchestrator → Intent → Agent → LLM/RAG/Tools → Services → PostgreSQL

Cette chaîne est composée de couches strictement découplées par interfaces (Clean Architecture) : le noyau IA (`src/ai/`) n'est jamais appelé directement par les routes HTTP — un unique point de jonction, le *Bridge* (`src/modules/ai-copilot/services/aiCopilotBridge.service.js`), fait la traduction entre le monde HTTP/Express et le monde de l'orchestrateur IA.

4. Frontend Flutter

Composant	Fichier	Rôle
CopilotFab	lib/ai/widgets/copilot_fab.dart	Bouton flottant, monté une seule fois au niveau d' AppShell (donc visible sur tout écran authentifié) ; masqué automatiquement quand le panneau est ouvert
CopilotChatDrawer	lib/ai/screens/copilot_chat_drawer.dart	Panneau latéral (desktop/tablette) contenant CopilotPanelBody
CopilotController	lib/ai/providers/copilot_controller.dart	État métier de la conversation (GetX) : envoi de message, streaming, gestion d'erreur, confirmation/annulation de PendingAction
CopilotUiController	lib/ai/providers/copilot_ui_controller.dart	État purement visuel (ouvert/fermé, onglet actif) — séparé délibérément de CopilotController pour qu'un changement d'onglet ne redéclenche jamais un rechargement de la conversation
CopilotPanelBody	lib/ai/screens/copilot_panel_body.dart	Corps partagé desktop/mobile : en-tête, onglets (dashboard/chat/historique), liste de messages, barre de saisie
CopilotResponse	lib/ai/models/copilot_response.dart	Modèle Dart reflétant 1:1 l'enveloppe JSON du ResponseBuilder backend (answer, intent, mode, citations, toolCalls, pendingAction, ...)
CopilotPendingActionCard	lib/ai/widgets/copilot_pending_action_card.dart	Affiche les étapes proposées et deux boutons (Confirmer/Annuler) qui envoient simplement les mots « confirme »/« annule » en langage naturel — aucun endpoint de confirmation dédié
ApiClient	lib/providers/api_client.dart	Client Dio unique et partagé de toute l'application, intercepteur JWT (Authorization: Bearer) déjà câblé, réutilisé par CopilotApiService plutôt qu'un second client
AuthService	lib/providers/auth_service.dart	Source du token (GetStorage , clé accessToken), écrit après connexion, lu par l'intercepteur Dio et par le service WebSocket du Copilot

5. Backend

Composant	Fichier	Rôle
AI Copilot Bridge	src/modules/ai-copilot/services/aiCopilotBridge.service.js	Seul point du backend métier qui importe directement <code>src/ai/</code> ; construit un orchestrateur par utilisateur, appelle <code>AIManager.handleMessage()</code>
AI Manager (léger)	src/ai/manager/aiManager.js — classe <code>AIManager</code>	Porte d'accès : <code>isEnabled()</code> , <code>canUseAiCopilot(user)</code> , <code>requireLlmProvider()</code> — jamais de logique de conversation
AI Manager (orchestrateur)	src/ai/orchestrator/aiManager.js — classe <code>AIManager</code>	Cerveau d'un tour de conversation complet : <code>handleMessage()</code> , 12 étapes (détaillées section 6)
Pipeline	src/ai/orchestrator/pipeline.js — <code>createAiOrchestrator()</code>	Composition racine : câble LangChain, RAG, MCP, sécurité, 19 agents, résolveurs de contexte
AgentRouter	src/ai/orchestrator/agentRouter.js	Une seule méthode <code>select(intent)</code> — délègue entièrement à <code>AgentRegistry.findHandler()</code>
AgentRegistry	src/ai/agents/agentRegistry.js	Enregistrement/recherche d'agent (<code>register/get/list/findHandler</code>) — pure plomberie, aucune logique RBAC
BaseAgent	src/ai/agents/baseAgent.js	Classe mère des 19 agents : vérifie RBAC (<code>isAuthorized</code>) avant toute exécution, construit l'instruction (prompt système + historique), choisit Tool Calling ou RAG
ResponseBuilder	src/ai/orchestrator/responseBuilder.js	Construit l'enveloppe JSON finale (<code>build/buildErrorResponse/buildClarificationResponse</code>) — jamais le message d'erreur brut exposé au client

6. Intent Detection

Keywords/scoring : `LexicalIntentRouter` (`langchain/router/lexicalIntentRouter.js`) compte, pour chaque route du catalogue, le nombre de mots-clés présents dans le message tokenisé — 100% déterministe, aucun appel LLM. **21 intentions** sont cataloguées (`intentCatalog.js`), de `documentation_question` à `general_chat` (repli par défaut).

Confidence : 4 paliers discrets (`intentConfidence.js`) — 0.30 (aucun mot-clé), 0.55 (égalité stricte hors documentation), 0.90 (override explicite ou gagnant net), 0.95 (égalité avec `documentation_question` , priorité procédurale).

Overrides : 7 mécanismes correctifs appliqués après le scoring lexical de base, chacun avec une condition étroite et un exemple réel vérifié par test :

Override	Exemple réel
<code>projectAnalysisOverride</code>	« pourquoi projet depot SBF est critique » → <code>projects</code>
<code>capabilityQuestionOverride</code>	« Qu'est-ce que je peux faire ? » → <code>capability_audit</code>
<code>projectOwnershipOverride</code>	« projets associés à un autre utilisateur » → <code>projects</code>
<code>projectDescribeOverride</code>	« détails de Résidence HOLYA » → <code>projects</code>
<code>commercialRelanceOverride</code>	« clients en retard de relance » → <code>commercial_contacts</code>
<code>commercialAssignmentOverride</code>	« affecter [contact] à email@... » → <code>commercial_contacts</code>
<code>industrialPromeshOverride</code>	« fiche PRO PROMESH » → <code>por_promesh</code> (jamais <code>industrial</code>)

Fallback : `general_chat` , dernière entrée du catalogue, capturant toute demande sans mot-clé reconnu.

Capability audit : intention dédiée (`capability_audit`) répondant à « qu'est-ce que je peux faire ? » via `RoleCapabilityAuditAgent` , qui interroge `CapabilityMatrixService` (déterministe, sans Tool ni LLM) pour lister les capacités réellement accessibles au rôle de l'utilisateur.

7. Multi-Agent

19 agents métiers réellement enregistrés (`agents/manifest.js` , `defaultAgentFactories`) :

Agent	Domaine	Intent	Tools	RAG	LLM	Écriture
AssistantAgent	Documentation générale	general_chat	0	Oui	Oui	Non
CrmAssistantAgent	Documentation CRM	documentation_question	0	Oui	Oui	Non
ProjectAgent	Projets/Pipeline	projects	4	Non	Partiel	Oui
CommercialAgent	Contacts commerciaux	commercial_contacts	6	Non	Partiel	Oui (PendingAction)
PipelineAgent	Étapes du pipeline	pipeline	6	Non	Oui	Oui
KpiAgent	Dashboard KPI	dashboard	4	Non	Non	Non
ReportingAgent	Rapports	reporting	4	Non	Oui	Non
MaintenanceAgent	Maintenance	maintenance	15	Non	Oui	Oui
IndustrialAgent	Fiches industrielles	industrial	5	Non	Partiel	Oui
NotificationAgent	Notifications	notifications	5	Non	Oui	Oui
AutomationAgent	Cross-module	automation	toutes (RBAC pré-filtré)	Non	Oui (Planner)	Oui (PendingAction)
BusinessAnalystAgent	Analyse d'affaires	business_analysis	6	Non	Non	Non
RecommendationAgent	Recommandations	recommendation	2	Non	Non	Non
DataQualityAgent	Qualité de données	data_quality	1 (lecture stricte)	Non	Non	Non
AdvancedSearchAgent	Recherche avancée	advanced_search	2	Non	Non	Non
MeetingAgent	Réunions/rappels	meetings, reminders	4	Non	Partiel	Oui (PendingAction)
RoleCapabilityAuditAgent	Audit de capacités	capability_audit	0	Non	Non	Non
HrAgent	Ressources humaines	hr	9	Non	Oui	Oui
PorPromeshAgent	POR PROMESH	por_promesh	10	Non	Partiel	Oui

« Partiel » (colonne LLM) : l'agent combine des chemins déterministes (sans LLM) pour les demandes fréquentes et un repli générique (LLM + `requireToolUse`) pour le reste. Un 20e agent (`SearchAgent`) existe dans le code mais n'est plus câblé par défaut (remplacé par `CrmAssistantAgent`) — exclu de ce tableau, qui ne recense que les agents réellement actifs.

8. LLM

Paramètre	Valeur vérifiée (Backend Master/.env réel)
Provider actif	Ollama (src/ai/services/providers/ollamaProvider.js)
Modèle	qwen3:8b — OLLAMA_MODEL=qwen3:8b
Endpoint	http://127.0.0.1:11434/api/chat (natif Ollama, pas le shim OpenAI)
Timeout	240000 ms (OLLAMA_TIMEOUT_MS), calibré au-dessus d'un pire cas réel mesuré de 179 681 ms
Plafond de génération	num_predict = 768 tokens (garde-fou anti-boucle, pas une troncature normale)
Fenêtre de contexte	num_ctx = 8192 tokens (envoyé explicitement, défaut silencieux Ollama mesuré à 4096)
Fallback inter-providers	NON TROUVÉ DANS LE CODE — aucune bascule automatique
Providers alternatifs implémentés	OpenAiProvider , AnthropicProvider — pleinement codés et testés, mais non actifs dans le .env actuel (ni modèle ni clé API renseignés)

Rôle du LLM dans l'architecture : le LLM n'est jamais la source de vérité sur une donnée métier ou sur l'exécution effective d'une action — il décide (format JSON structuré) d'appeler un outil ou de répondre, et cette décision est systématiquement révérifiée par des mécanismes indépendants (steps.length réel, RBAC, validation Joi) avant d'affecter quoi que ce soit. Le contrat unique ILLmProvider.complete(messages, options) → string découple entièrement le reste du système du fournisseur concret.

9. Prompt Engineering

Couche	Contenu / rôle
System Prompt (générique)	<code>buildResponseSystemPrompt()</code> (<code>responsePrompts.js</code>) : principes anti-hallucination + citation obligatoire des sources, plus un ton adapté à 5 rôles (admin, commercial, rh, maintenance, direction)
System Prompt (par agent)	20 agents possèdent leur propre constante <code>SYSTEM_PROMPT</code> métier (ex. interdiction des balises HTML pour <code>PorPromeshAgent</code> , une seule question à la fois pour <code>MeetingAgent</code>)
User Prompt	Le message brut de l'utilisateur, éventuellement scanné par <code>PromptInjectionGuard</code> avant traitement
Context	<code>RagContextBuilder.buildContext()</code> assemble citations RAG + historique de conversation en une chaîne unique injectée au prompt (<code>QaChain</code>)
Tool instructions	<code>buildSystemPrompt(tools, role)</code> (<code>toolCallingChain.js</code>) liste les outils disponibles et impose un protocole JSON strict à deux formes : <code>call_tool</code> ou <code>final_answer</code>
Output instructions	<code>RESPONSE_PRINCIPLES</code> : réponse professionnelle, courte, jamais de donnée inventée, citation systématique des sources/outils
Security	Le prompt n'est jamais traité comme un mécanisme de sécurité — voir raisonnement ci-dessous

Pourquoi un prompt système n'est jamais une protection RBAC : le code lui-même le documente explicitement pour `CrmAssistantAgent` — « la garantie "ne jamais inventer" vient de cette architecture [`SourceGuard`], pas d'une promesse du prompt seul » (`crmAssistantAgent.js`). Le RBAC réel est vérifié en code (`isAuthorized()`) avant tout appel LLM et à nouveau avant chaque exécution d'outil (`PermissionManager`) — indépendamment de ce que le modèle a pu lire, halluciner ou ignorer dans le prompt.

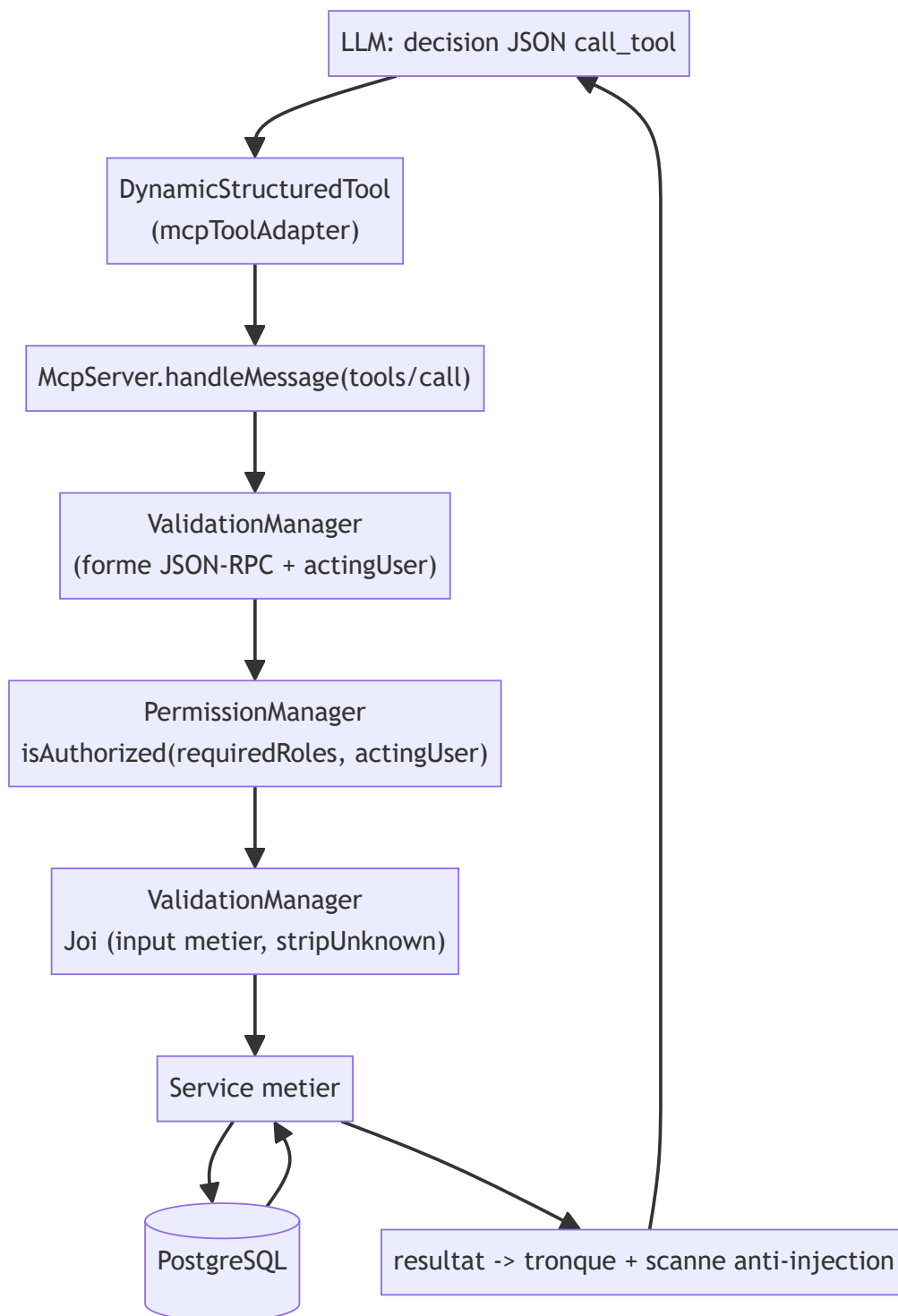
10. RAG



Figure — Pipeline RAG réel (aucune technologie supposée : embedding par hachage, pas de modèle neuronal ; stockage en mémoire, pas de base vectorielle persistante)

Étape	Technologie réelle vérifiée
Documents	docs/ , README interne, AI-Business-Copilot/ , un PDF d'architecture — jamais les données métier CRM
Ingestion	Scan récursif générique, racines explicites obligatoires (aucun défaut implicite)
Chunking	800 caractères, overlap 120, découpage par paragraphes
Embedding	Hachage déterministe (feature hashing signé), 256 dimensions — PAS un modèle d'embedding neuronal
Vector store	En mémoire volatile (Map () JS) — PAS une base vectorielle persistante (pas de pgvector, Pinecone, Weaviate ni Qdrant trouvés dans le code)
Retrieval	Similarité cosinus, calcul brute-force (pas d'index ANN)
Reranking	Similarité de Jaccard + facteur de catégorie par chemin de document (pas de cross-encoder)
Context	RagContextBuilder assemble citations + historique, avec scan anti-injection
LLM	QaChain — répond exclusivement à partir du contexte, jamais des données métier
Citations	6 champs exposés au frontend (index, title, sourceType, path, excerpt, score)

11. Tool Calling



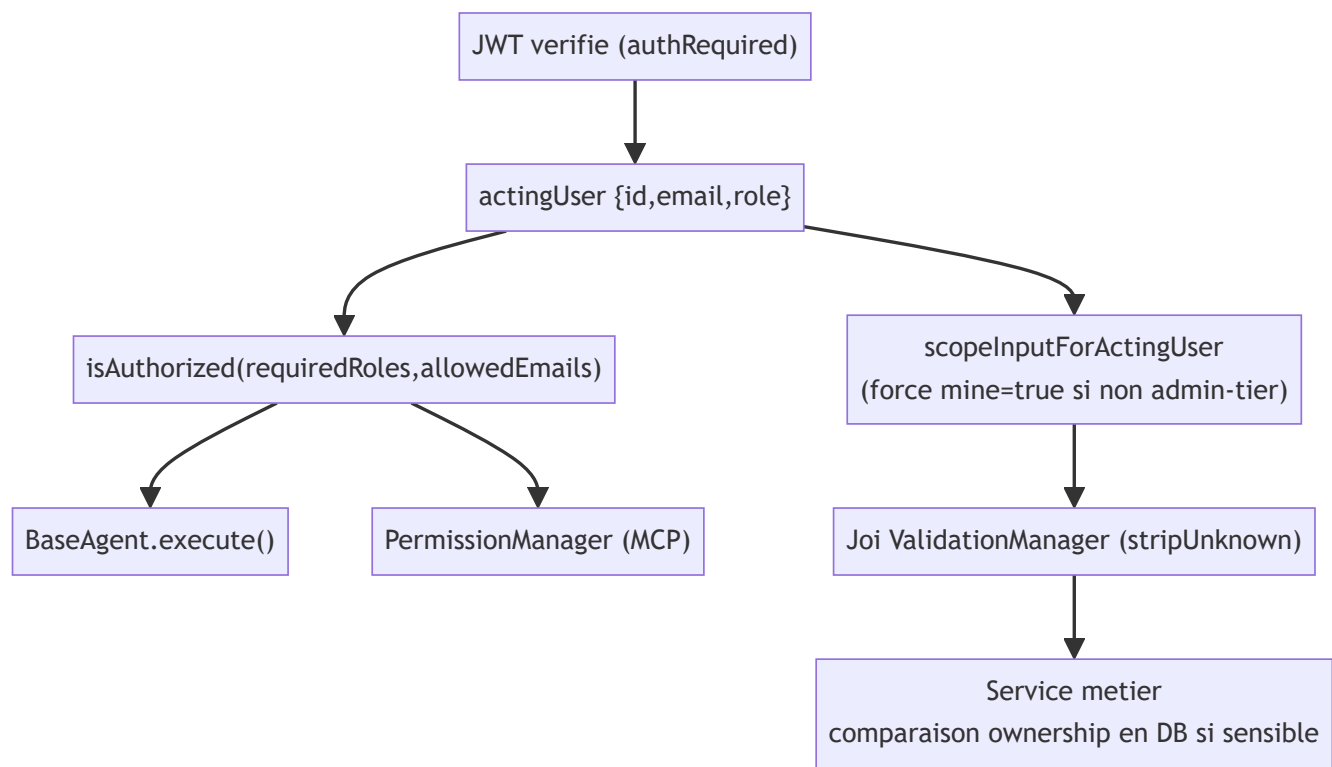
Cycle vérifié dans `langchain/chains/toolCallingChain.js` et `mcp/server/mcpServer.js` : le LLM ne peut jamais contourner Validation/Permission — même un appel halluciné avec des paramètres inventés est rejeté avant d'atteindre le service métier. Boucle bornée à `maxToolCalls=5` (garde-fou anti-boucle infinie).

12. MCP

Un protocole JSON-RPC 2.0 réel et conforme à la spécification MCP est implémenté (`initialize` , `ping` , `tools/list` , `tools/call` , transport stdio réel) — mais il est utilisé quasi exclusivement **en process** dans l'application réelle (le serveur MCP est instancié comme un objet JavaScript ordinaire et appelé en mémoire, jamais via un second processus). Le seul chemin où MCP tourne comme un vrai serveur séparé (`node src/ai/mcp/server/start.js`) n'est invoqué par aucun script npm de production.

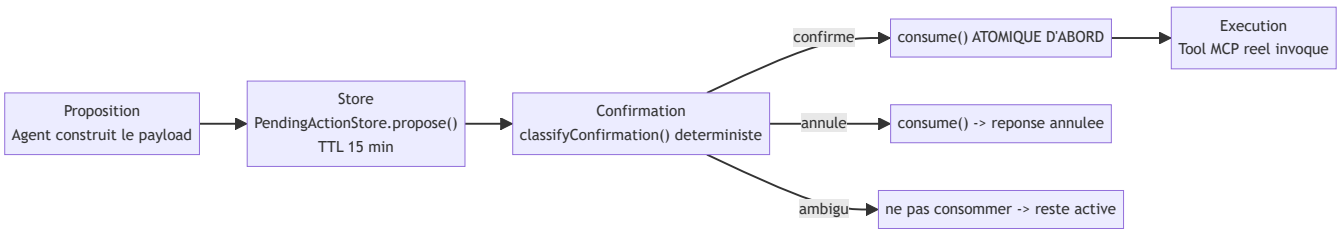
76 Tools réels répartis sur 12 domaines (comptage vérifié par exécution du code, et non par les commentaires internes qui citent des chiffres obsolètes de 69/73/74). Chaque appel de Tool traverse systématiquement `PermissionManager` puis `ValidationManager` avant d'atteindre le service métier — même chemin qu'un client MCP externe respecterait, qu'il soit invoqué depuis `ToolCallingChain` (LangChain) ou depuis la confirmation d'un workflow proactif.

13. Sécurité



Mécanisme	Rôle vérifié
JWT	Seule source de <code>role / email / id</code> — jamais dérivé d'un message ou d'une décision LLM
RBAC (<code>isAuthorized</code>)	Fonction unique, partagée par Agents et MCP — 7 rôles réels (<code>user</code> , <code>commercial</code> , <code>accueil</code> , <code>admin</code> , <code>superadmin</code> , <code>superadmin2</code> , <code>responsable_logistique_achat</code>); <code>accueil</code> exclu de l'accès Copilot, <code>finance_probar</code> inexistant
<code>actingUser</code>	Objet normalisé <code>{id,email,role}</code> , seul support transmis à l'orchestrateur — jamais le payload JWT brut
Ownership	Comparaison explicite <code>createdBy===actingUser.id</code> pour les relances commerciales ; deux exceptions identifiées (<code>assignOwner</code> , <code>moveStage</code>) où seul le rôle protège, sans revérification en base
Joi	Validation stricte des schémas d'entrée de chaque Tool, <code>stripUnknown</code> — un champ d'ownership halluciné par le LLM et non déclaré est rejeté silencieusement avant le service
PermissionManager	Vérifié AVANT la validation métier de l'input — un rôle non autorisé n'atteint jamais l'exécution du Tool
<code>scopeInputForActingUser</code>	Force le périmètre de lecture à l'utilisateur courant pour tout rôle non admin-tier, quel que soit ce que le LLM a fourni
Prompt Injection	5 règles regex (FR/EN), 3 modes (flag/sanitize/block) — ne capture aucune des attaques de type « injection de paramètre » (<code>ownerId=...</code>), neutralisées par des mécanismes indépendants (JWT, scoping, Joi)
Anti-fuite	Incident réel documenté et corrigé : pour « projets associés à Hadil », le LLM a produit un champ halluciné ignoré par erreur, exposant 5/20 projets d'autres utilisateurs avant le correctif <code>scopeInputForActingUser</code>

14. PendingAction



Agent	Qui choisit le payload	Étapes exposées	Rôles
CommercialAgent	100% résolution serveur (jamais le LLM)	1 (relance)	commercial, admin-tier
MeetingAgent	100% extraction déterministe (regex)	1 (réunion)	ouvert (scope forcé par Tool)
AutomationAgent	Le LLM, via LlmPlanner (seul cas)	N (arbitraire, cross-module)	admin-tier uniquement

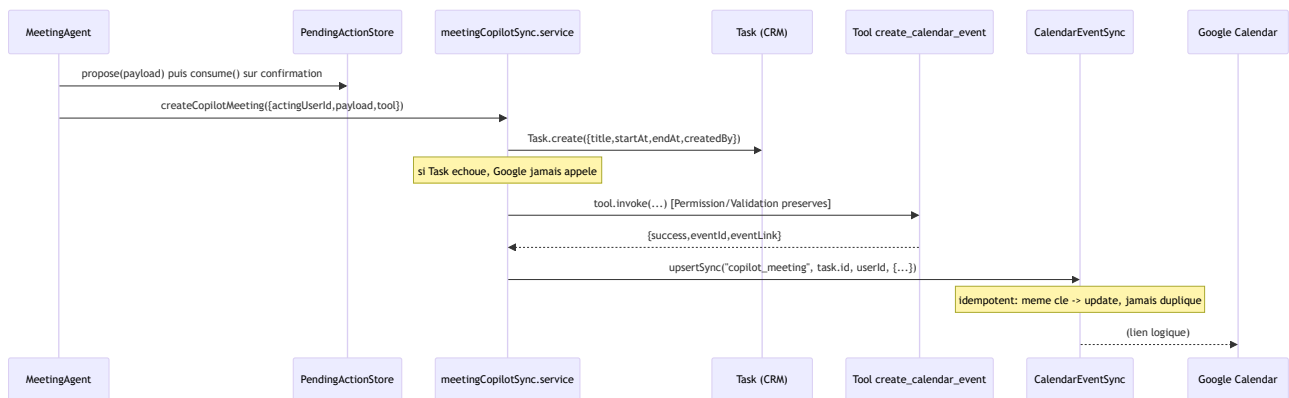
Contrat commun : `{id, steps:[{tool,input,reason}]}` . Flutter ne renvoie jamais cet objet au serveur — la confirmation transite par le mot « confirme »/« annule » en langage naturel, et l'exécution relit toujours le payload stocké côté serveur. `consume()` atomique élimine toute double exécution, vérifié par test réel (2e confirmation → « aucune action en attente »).

15. Cas d'utilisation

#	Question	Intent	Agent	Tool/RAG	Résultat
1	« Donne-moi la liste des projets »	projects	ProjectAgent	Tool: list_projects	Liste scopée à l'utilisateur (ou globale si admin-tier)
2	« Quels sont mes KPI ce mois-ci ? »	dashboard	KpiAgent	Tool: get_dashboard_kpis (déterministe)	Chiffres réels du Tool, jamais devinés
3	« Quels sont mes contacts commerciaux ? »	commercial_contacts	CommercialAgent	Tool: get_my_commercial_contacts	Liste scopée RBAC
4	« Relancer [contact] pour le 27/08 et affecter [email] »	commercial_contacts	CommercialAgent	Tool: schedule_commercial_contact_relance (PendingAction)	Relance créée + synchro Task/Google (après confirmation)
5	« Programmer une réunion avec [email] demain à 10h »	meetings	MeetingAgent	Tool: create_calendar_event (PendingAction)	Task CRM + événement Google + CalendarEventSync
6	« Donne-moi les fiches POR PROMESH »	por_promesh	PorPromeshAgent	Tool: list_por_promesh (chemin déterministe ou requireToolUse)	Fiches réelles, jamais un final_answer sans appel réel
7	« Je veux faire une demande de congé »	hr	HrAgent	Tool: create_hr_request	Demande RH créée pour l'utilisateur courant
8	« Il y a une panne sur le site »	maintenance	MaintenanceAgent	Tool: create_maintenance_request	Demande de maintenance créée
9	« Marque mes notifications comme lues »	notifications	NotificationAgent	Tool: mark_all_notifications_read	Notifications de l'utilisateur mises à jour
10	« Crée une demande de maintenance et confirme »	automation	AutomationAgent	Planner → Tool(s) proposés (PendingAction)	Exécution cross-module après confirmation explicite

Un cas documentaire (« Comment créer un projet ? ») aurait pu être ajouté comme 11e exemple RAG pur (Intent: documentation_question → CrmAssistantAgent → RAG, sans Tool) — non compté dans les 10 demandés mais pertinent pour illustrer la distinction RAG/Tool Calling déjà démontrée dans l'audit LLM/RAG précédent.

16. Google Calendar



Migration `copilot_meeting` : l'ENUM `entityType` de la table `calendar_event_syncs` a été étendu par la migration `20260815120000-add-copilot-meeting-to-calendar-event-syncs-enum.js` pour tracer spécifiquement les réunions créées depuis le Copilot conversationnel — distinctes des entités déjà existantes (`project_action` , `commercial_contact_reliance`). Cette migration a été nécessaire car le mécanisme `createCopilotMeeting()` réutilise le système générique `CalendarEventSync / multiRecipientCalendarSync.service.js` déjà en place pour les autres entités, plutôt que de créer une seconde implémentation.

Idempotence vérifiée par test réel (`services.calendarEventSync.copilotMeeting.test.js`) : un second `upsertSync` sur exactement le même (`entityType`, `entityId`, `userId`) met à jour la ligne existante, jamais un doublon.

17. Base de données

Agent	Tool (exemple)	Service	Modèle	Table
ProjectAgent	list_projects, assign_project_owner, move_project_stage	project.service.js	Project	projects
ProjectAgent	create_project_action (via ProjectActions)	projectAction.service.js	ProjectAction, ProjectReminder	project_actions, project_reminders
CommercialAgent	get_my_commercial_contacts, schedule_commercial_contact_reliance	commercialContactRelance.service.js, commercialContactKpi.service.js	CommercialContact, CommercialContactRelance	commercial_contacts, commercial_contact_relances
HrAgent	create_hr_request	hrRequest.service.js	HrRequest, HrRequestActivity	hr_requests, hr_request_activities
MaintenanceAgent	create_maintenance_request, assign_maintenance_technician	maintenanceRequest.service.js	MaintenanceRequest	maintenance_requests
IndustrialAgent	create_industrial_record	(service industriel, module industrial-records)	modèle industriel dédié	industrial_records (nom exact non revérifié dans cet audit)
PorPromeshAgent	list_por_promesh, validate_por_promesh	porPromesh.service.js	PorPromesh	por_promesh
NotificationAgent	list_my_notifications, send_notification_email	notification.service.js	Notification	notifications
KpiAgent / ReportingAgent	get_dashboard_kpis	dashboard.service.js	agrégats multi-tables	projects, commercial_contacts, maintenance_requests...
MeetingAgent	create_calendar_event	googleCalendar.service.js, meetingCopilotSync.service.js	Task, CalendarEventSync	tasks, calendar_event_syncs
PipelineAgent	list_pipeline_stages, create_pipeline_stage	pipelineStage.service.js	PipelineStage	pipeline_stages
(RBAC transversal)	set_user_role, get_my_profile	user.service.js	User	users

Le nom exact du service et du modèle Sequelize sous-jacent à `create_industrial_record` n'a pas été revérifié dans le détail au cours de cet audit (seul le Tool et son agent ont été confirmés dans les rapports précédents) — « **Cette information n'a pas été trouvée dans le code analysé** » pour le nom exact de la table ; à vérifier avant dépôt.

18. Tests

Catégorie	Fichiers représentatifs
Unitaires — Intent	ai.orchestrator.intentDetector.test.js, ai.orchestrator.intentConfidence.test.js
Unitaires — Agents	ai.agents.*.test.js (un fichier par agent), ai.agents.manifest.test.js
Intégration	ai.orchestrator.commercialAgent.relanceAction.integration.test.js, ai.orchestrator.meetingAgent.liveCreate.integration.test.js, ai.orchestrator.porPromeshAgent.integration.test.js, ai.orchestrator.hrAgent.integration.test.js
DB	ai.mcp.tools.getPersonalKpiSummary.dbRegression.test.js, services.calendarEventSync.copilotMeeting.test.js
Live (contre un vrai serveur Ollama)	ai.agents.porPromeshAgent.htmlFree.live.test.js
RBAC	ai.copilotPermissions.test.js (7 rôles réels testés individuellement), ai.mcp.permissionManager.test.js
Anti-fuite	ai.orchestrator.projectOwnershipOverride.test.js, ai.mcp.tools.projects.test.js
Idempotence	services.calendarEventSync.copilotMeeting.test.js (upsertSync, count==1 après 2 appels)
Confirmation/Annulation	ai.agents.meetingAgent.confirmationGate.test.js, ai.automation.confirmationFlow.test.js
Corpus de validation	test/fixtures/copilotValidationScenarios.js — 500+ scénarios réels par rôle/intent
Architecture (garantie mécanique)	ai.mcp.tools.policy.test.js — échoue la build si un Tool accède directement à Sequelize/SQL

Les **résultats numériques d'exécution** (nombre exact de suites/tests passés) cités dans un document interne (`08_LLM_PROVIDER_OPENAI.md` : « 156 suites, 4721 tests — 100% verts ») datent d'une passe antérieure du projet et n'ont pas été ré-exécutés dans le cadre de cet audit — cité comme référence documentaire historique, pas comme un résultat vérifié à la date du 29 août 2026.

19. Problèmes rencontrés et corrections

19.1 Fuite de portée — list_projects / RBAC des Tools de lecture

PROBLÈME	Un rôle non admin-tier pouvait recevoir des projets d'autres utilisateurs
CAUSE	Pour « donne-moi les projets associés à Hadil », qwen3:8b a produit {"filter_owner":"Hadil"} (champ halluciné), silencieusement ignoré par un schéma Joi permissif, retombant sur mine=false (aucun filtre)
SOLUTION	scopeInputForActingUser() supprime mine/myProjects/ownerId/userId de l'input pour tout rôle non admin-tier et force mine:true
FICHIERS	mcp/tools/projects/listProjectsTool.js, même discipline dans getMyCommercialContactsTool.js
TEST	ai.mcp.tools.projects.test.js, ai.orchestrator.projectOwnershipOverride.test.js
RÉSULTAT	Résultat mesuré avant correctif : 5/20 projets appartenant à d'autres utilisateurs ; après correctif, périmètre toujours forcé côté serveur

19.2 get_commercial_personal_kpi_summary — RBAC des KPI personnels

PROBLÈME	Risque d'accès à des KPI personnels d'un autre commercial
CAUSE	Référencé comme précédent d'un « correctif RBAC P0 » dans les commentaires d'autres Tools du même module (getMyCommercialContactsTool.js : « même discipline que le correctif RBAC P0 de getPersonalKpiSummaryTool.js »)
SOLUTION	Scoping par ADMIN_TIER_ROLES (getPersonalKpiSummaryTool.js:26)
FICHIERS	mcp/tools/commercialContacts/getPersonalKpiSummaryTool.js
TEST	Détail exact du test non revérifié dans cet audit – « Cette information n'a pas été trouvée dans le code analysé » pour la référence précise du fichier de test
RÉSULTAT	Pattern ADMIN_TIER_ROLES confirmé en place dans le fichier actuel

19.3 Notifications — route legacy non authentifiée vs Tool restreint

PROBLÈME	La route HTTP legacy équivalente à l'envoi d'email de notification n'exige AUCUNE authentification (faille P0 documentée)
CAUSE	Héritage d'une route ancienne du CRM, hors périmètre de cette correction
SOLUTION	Le Tool MCP send_notification_email est délibérément plus restrictif que cette route : requiredRoles: ADMIN_TIER_ROLES
FICHIERS	mcp/tools/notifications/sendNotificationEmailTool.js:11-13
TEST	ai.mcp.tools.notifications.test.js
RÉSULTAT	Le Tool ne reproduit jamais la faiblesse de la route legacy, indépendamment de sa correction éventuelle

19.4 moduleAccessGuard — accès Copilot bloqué pour responsable_logistique_achat

PROBLÈME	Ce rôle recevait un 403 sur CHAQUE message Copilot, avant même l'exécution de l'IntentDetector
CAUSE	La liste de préfixes autorisés (ALLOWED_PREFIXES_BY_ROLE) n'avait jamais été mise à jour quand /ai-copilot a été monté dans app.js, alors que ce rôle avait déjà un accès Copilot réel et documenté (industrial/maintenance/por_promesh/hr)
SOLUTION	Ajout de /ai-copilot à la liste des préfixes autorisés pour ce rôle
FICHIERS	src/middleware/moduleAccessGuard.js:8-27
TEST	ai.copilotPermissions.test.js (rôle testé explicitement comme autorisé)

RÉSULTAT	Porte HTTP grossière et gate applicative désormais alignées — seul l'accès à la route est ajouté, aucune permission supplémentaire
-----------------	--

19.5 `capabilityQuestionOverride` / `projectOwnershipOverride` — routage d'intention incorrect

PROBLÈME	Des questions sur les capacités du Copilot (« qu'est-ce que je peux faire ? ») ou sur la propriété d'un projet étaient mal routées par le scoring lexical de base
CAUSE	Chevauchement de mots-clés avec d'autres intentions (<code>general_chat</code> , <code>users</code>)
SOLUTION	Overrides ciblés, appliqués après le scoring de base, avec condition étroite
FICHIERS	<code>orchestrator/capabilityQuestionOverride.js</code> , <code>orchestrator/projectOwnershipOverride.js</code>
TEST	Corpus de validation (<code>copilotValidationScenarios.js</code>), tests unitaires dédiés par override
RÉSULTAT	Exemples vérifiés : « Qu'est-ce que je peux faire ? » → <code>capability_audit</code> ; « projets associés à un autre utilisateur » → <code>projects</code>

19.6 `HrAgent` et `PorPromeshAgent` — extension de couverture métier

PROBLÈME	Les domaines RH et POR PROMESH n'étaient pas couverts par un agent conversationnel dédié
CAUSE	Extension progressive du périmètre du Copilot (agents ajoutés en 18e et 19e position, <code>manifest.js</code>)
SOLUTION	Ajout de <code>HrAgent</code> (9 tools) et <code>PorPromeshAgent</code> (10 tools)
FICHIERS	<code>agents/hrAgent.js</code> , <code>agents/porPromeshAgent.js</code> , <code>agents/manifest.js</code>
TEST	<code>ai.orchestrator.hrAgent.integration.test.js</code> , <code>ai.orchestrator.porPromeshAgent.integration.test.js</code>
RÉSULTAT	19 agents actifs au total, RBAC propre à chaque domaine

19.7 `requireToolUse` — hallucination de réponse sans appel d'outil

PROBLÈME	Pour « donner moi les fiches produites PROMESH », le modèle répondait un <code>final_answer</code> dès le premier tour (<code>toolCalls:[]</code> , <code>completion=75</code> tokens) sans jamais appeler <code>list_por_promesh</code> ; cas analogue pour <code>ProjectAgent</code> (« Donne-moi les projets associés à Hadil » → le texte prétendait avoir utilisé l'outil sans que <code>steps</code> le confirme) et pour <code>CommercialAgent</code> (« Combien ai-je de contacts ? » → réponse « 12 » entièrement fabriquée)
CAUSE	Le suivi du protocole JSON par un LLM local (qwen3:8b) n'est pas fiable — son propre texte ne peut jamais servir de preuve qu'un outil a réellement été appelé
SOLUTION	<code>requireToolUse: true</code> — rejette tout <code>final_answer</code> si <code>steps.length===0</code> , fondé exclusivement sur l'état réel de la boucle d'exécution
FICHIERS	<code>langchain/chains/toolCallingChain.js</code> (mécanisme), <code>porPromeshAgent.js:409</code> , <code>projectAgent.js:680</code> , <code>commercialAgent.js:671</code> (activation)
TEST	<code>test/ai.agents.porPromeshAgent.test.js:186-198</code>
RÉSULTAT	Message honnête systématique (« Je n'ai pas pu récupérer les données CRM nécessaires... ») plutôt qu'une hallucination plausible

19.8 `PendingAction` — contrat non uniforme et absence de garantie d'exécution unique

PROBLÈME	(P1.1) Aucune garde ne protégeait <code>PorPromeshAgent</code> / <code>ProjectAgent</code> contre une exécution non vérifiée ; (P1.2) la forme exposée à l'API/Flutter différait selon l'agent ; (P1.3) risque de duplication d'une synchronisation calendrier rejouée
CAUSE	Évolution progressive du mécanisme <code>PendingAction</code> , initialement conçu pour <code>AutomationAgent</code> seul
SOLUTION	(P1.1) <code>requireToolUse</code> ; (P1.2) normalisation du contrat <code>{id, steps:[{tool,input,reason}]}</code> , identique pour les 3 agents ; (P1.3) idempotence de <code>upsertSync</code> par (<code>entityType</code> , <code>entityId</code> , <code>userId</code>)

FICHIERS	<code>orchestrator/pendingActionStore.js</code> , <code>commercialAgent.js</code> , <code>meetingAgent.js</code> , <code>automationAgent.js</code> , <code>multiRecipientCalendarSync.service.js</code>
TEST	<code>test/services.calendarEventSync.copilotMeeting.test.js:73-83</code> (idempotence, mission « P1.3 »)
RÉSULTAT	Un second <code>upsertSync</code> sur la même clé met à jour la ligne existante, jamais un doublon

19.9 CalendarEventSync — traçabilité des réunions créées par le Copilot

PROBLÈME	Aucune valeur d'ENUM ne permettait de distinguer une réunion créée par le Copilot des entités déjà tracées (actions de projet, relances)
CAUSE	Extension du système générique de synchronisation calendrier à un nouveau type d'entité
SOLUTION	Migration ajoutant la valeur <code>copilot_meeting</code> à l'ENUM <code>entityType</code>
FICHIERS	<code>src/migrations/20260815120000-add-copilot-meeting-to-calendar-event-syncs-enum.js</code>
TEST	<code>services.calendarEventSync.copilotMeeting.test.js</code>
RÉSULTAT	Réutilisation du mécanisme générique existant, jamais une seconde implémentation

19.10 Synchronisation calendrier — Task CRM + Google Calendar pour les relances commerciales

PROBLÈME	Une relance programmée par le Copilot n'apparaissait ni dans le calendrier CRM ni dans Google Calendar
CAUSE	Le Tool <code>schedule_commercial_contact_relance</code> se limitait initialement à l'écriture en base
SOLUTION	Réutilisation de <code>multiRecipientCalendarSync.service.js</code> (même mécanisme que <code>followupAutomation.service.js</code> et <code>meetingCopilotSync.service.js</code>) via <code>_syncCalendars()</code> , best-effort, jamais bloquant pour l'écriture déjà réussie de la relance
FICHIERS	<code>commercialContactRelance.service.js:62-148</code>
TEST	Test spécifique non identifié précisément dans cet audit – « Cette information n'a pas été trouvée dans le code analysé » pour un fichier de test dédié à <code>_syncCalendars</code>
RÉSULTAT	Une relance reste écrite même si la synchronisation calendrier échoue (reflété honnêtement via <code>googleCalendarSynced</code> / <code>googleCalendarError</code>)

19.11 RoleCapabilityAuditAgent — absence de réponse fiable à « que puis-je faire ? »

PROBLÈME	Avant cet agent, une question sur les capacités du rôle courant n'avait pas de chemin déterministe dédié
CAUSE	Absence d'un service de référence unique listant les capacités par rôle
SOLUTION	Ajout de <code>RoleCapabilityAuditAgent</code> (19e agent), lecture seule stricte, s'appuyant sur <code>CapabilityMatrixService</code> (déterministe, sans LLM ni Tool)
FICHIERS	<code>agents/roleCapabilityAuditAgent.js</code> , <code>reasoning/capabilityMatrixService.js</code>
TEST	<code>ai.agents.roleCapabilityAuditAgent.test.js</code> , <code>ai.reasoning.capabilityMatrixService.test.js</code>
RÉSULTAT	Réponse cohérente avec les restrictions RBAC réelles de l'utilisateur, jamais une réponse générique

20. Performance

Toutes les valeurs de cette section proviennent de constantes de configuration ou de mesures RÉELLES citées littéralement dans les commentaires du code (issues de diagnostics de production documentés) — aucun benchmark systématique n'a été trouvé dans le dépôt.

Mesure	Valeur
LLM — pire cas mesuré (contention CPU, sans GPU)	179 681 ms (prompt=2762 tokens, completion=227 tokens)
LLM — même appel sans contention	9 084 ms (facteur ~20× sur le même travail)
Timeout configuré	240 000 ms (marge au-dessus du pire cas mesuré)
Nombre d'appels LLM par tour « légitime »	2 appels (mesuré pour PorPromeshAgent)
Plafond de boucle Tool Calling	5 tours maximum (<code>maxToolCalls</code>)
Retry	2 tentatives maximum (1 réelle + 1 retry), 500 ms d'attente
Taille de résultat d'outil réinjecté	Tronqué à 4000 caractères (mesuré : un résultat non tronqué atteignait 42 115 caractères)
Fenêtre de contexte envoyée	8192 tokens (vs défaut silencieux Ollama à 4096)
Latence RAG (embedding + recherche + reranking)	NON TROUVÉ DANS LE CODE — aucune mesure chiffrée dédiée
Latence DB (requêtes Sequelize)	NON TROUVÉ DANS LE CODE — aucune mesure chiffrée dédiée

21. Limites

Limites actuelles

- Ownership non revérifié dans les services `assignOwner()` / `moveStage()` — seul le rôle protège
- `ADMIN_TIER_ROLES` dupliqué dans 8 fichiers plutôt qu'une constante centralisée
- État conversationnel et `PendingAction` en mémoire process, non partagés entre instances, perdus au redémarrage

Fonctionnalités non exposées

- Le Tool `create_calendar_event` n'a pas de champ « participants » — jamais promis par le prompt système de `MeetingAgent`, qui l'interdit explicitement
- Aucun Tool ne permet de lister les réunions/rappels déjà existants (confirmé absent, documenté dans le `SYSTEM_PROMPT` de `MeetingAgent` lui-même)

Fonctionnalités non implémentées

- Streaming token-par-token du LLM (`ILlmProvider.stream()`) jamais surchargé
- Détection de conflit d'horaires calendrier — `NON TROUVÉ DANS LE CODE` (recherche explicite effectuée, aucune occurrence)
- Base vectorielle persistante (pgvector ou équivalent) — actuellement en mémoire volatile
- Fallback automatique inter-providers LLM

Problèmes connus

- Documentation interne partiellement désynchronisée du code (nombre d'intentions, nombre de Tools)
- Composants orphelins présents mais non câblés en production (`IntentClassifier`, `SearchAgent`)

Limites du LLM

- Modèle local (qwen3:8b, CPU seul) : variance de latence extrême (facteur $\sim 20\times$ observé)
- Suivi imparfait du protocole de tool-calling, compensé mais non éliminé par `requireToolUse` (activé sur 3 agents seulement à ce jour)

Limites RAG

- Embedding par hachage — aucune similarité sémantique réelle au-delà du recouvrement de tokens
- Stockage vectoriel volatile, recherche brute-force sans index approximatif

Limites Google Calendar

- Synchronisation best-effort, jamais transactionnelle avec l'écriture CRM (limite assumée et documentée, pas un bug)
- Aucune gestion de conflit d'horaires

22. Perspectives

Cette section propose des améliorations **non implémentées** — clairement distinguées de l'état actuel décrit dans les sections précédentes. Elles sont formulées à partir des points d'extension explicitement anticipés par les interfaces du code (`IEembeddingProvider` , `IVectorStore` , `ILlmProvider`).

Perspective	État actuel (rappel)	Piste d'évolution
Meilleure mémoire conversationnelle	En mémoire process, volatile, non partagée	Persistance externe (Redis déjà utilisé ailleurs dans le kernel pour le rate limiting)
RAG persistant	Embedding par hachage, stockage en mémoire	Adaptateur <code>IEembeddingProvider</code> neuronal + <code>IVectorStore</code> <code>pgvector</code>
Meilleure observabilité	Logs textuels par namespace, pas de tableau de bord dédié — NON TROUVÉ DANS LE CODE de dashboard de monitoring LLM	Tableau de bord de suivi des latences/erreurs par provider
Streaming	Non implémenté (<code>stream()</code> jamais surchargé)	Implémentation pour Ollama (supporte nativement <code>stream:true</code>)
Caching	Aucun mécanisme de cache de réponse LLM trouvé dans le code	Cache des réponses pour les questions documentaires répétées (RAG)
Nouveaux Agents	19 agents actifs, domaines couverts listés section 7	Extension à d'autres domaines du CRM non encore couverts
Gestion des conflits calendrier	Aucune détection de conflit	Vérification de chevauchement avant confirmation de création
Monitoring LLM	Logs de tokens/latence par appel, jamais agrégés	Agrégation et alerting sur la latence/le taux d'échec par provider
Évaluation automatique	Un corpus de 500+ scénarios existe et s'exécute via les tests (<code>copilotValidationScenarios.js</code>) — une forme de régression automatisée existe déjà	Évaluation continue en production (LLM-juge, scoring qualité de réponse)

23. Questions du jury

55 questions/réponses, fondées exclusivement sur le projet réel et les trois audits de code réalisés.

LLM (5)

Q1. Quel LLM est réellement utilisé en production dans ce projet ?

Ollama, servant le modèle `qwen3:8b` en local, sans clé API — confirmé par le fichier `.env` réel (`AI_LLM_PROVIDER=ollama`).

Q2. Pourquoi le streaming n'est-il pas utilisé alors qu'Ollama le supporte ?

Le contrat `ILlmProvider.complete()` attend une chaîne unique résolue ; aucun adaptateur du kernel ne surcharge `stream()` — implémenter le streaming nécessiterait de changer ce contrat et tous ses consommateurs (LangChain, agents).

Q3. Le système peut-il basculer automatiquement vers un autre provider en cas de panne ?

Non — le retry ne rejoue que le même provider (2 tentatives maximum) ; aucune bascule automatique inter-providers n'existe dans le code.

Q4. Pourquoi le timeout Ollama est-il si élevé (240 secondes) ?

Un pire cas réel mesuré à 179 681 ms (contention CPU, absence de GPU) a été observé en diagnostic ; le timeout a été fixé au-dessus de cette valeur pour ne jamais couper un appel légitime en cours.

Q5. OpenAI et Anthropic sont-ils utilisés dans ce projet ?

Les deux adaptateurs (`OpenAiProvider`, `AnthropicProvider`) sont pleinement implémentés et testés, mais non actifs dans la configuration actuelle — seul un provider est actif à la fois.

RAG (4)

Q6. Quel modèle d'embedding est utilisé pour le RAG ?

Aucun modèle neuronal — un algorithme de hachage déterministe (feature hashing signé), 256 dimensions, entièrement hors ligne.

Q7. Où sont stockés les vecteurs du RAG ?

En mémoire du processus Node (`Map()` JavaScript), jamais dans une base vectorielle persistante — perdu à chaque redémarrage du serveur.

Q8. Le RAG peut-il accéder aux données métier du CRM ?

Non, structurellement : `SourceGuard` impose une allowlist d'extensions documentaires et une blocklist de fragments de chemin métier, vérifiée même contre les symlinks.

Q9. Comment le reranking fonctionne-t-il sans modèle ML ?

Par similarité de Jaccard (recouvrement de tokens) combinée au score cosinus, pondérée par un facteur de catégorie dérivé du chemin du document (ex. guide utilisateur boosté, documentation interne du Copilot pénalisée).

Embeddings (3)

Q10. Quelle est la dimension des vecteurs d'embedding ?

256, configurable via `RAG_EMBEDDING_DIMENSIONS`.

Q11. Le hachage utilisé est-il sécurisé cryptographiquement ?

Non, ce n'est pas son objectif — c'est un hachage de type djb2 utilisé pour répartir des tokens dans un espace vectoriel de faible dimension (feature hashing), pas un hachage cryptographique.

Q12. Que se passerait-il si on remplaçait cet embedding par un vrai modèle neuronal ?

Un seul adaptateur (`IEmbeddingProvider`) serait à remplacer — l'architecture en ports/adaptateurs garantit que `RagRetrievalService` / `RagContextBuilder` n'auraient pas à changer.

Vector Store (3)

Q13. Quelle technologie de base vectorielle est utilisée ?

Aucune base dédiée — un `Map()` JavaScript en mémoire (`InMemoryVectorStore`), avec recherche cosinus brute-force.

Q14. Cette solution passerait-elle à l'échelle sur un corpus de plusieurs milliers de documents ?

Non — la recherche est en $O(n)$ sans index approximatif ; c'est une limite explicitement documentée dans le code comme un choix de phase de développement.

Q15. Le port `IVectorStore` anticipe-t-il un remplacement futur ?

Oui — le commentaire du fichier interface indique explicitement qu'un adaptateur persistant (pgvector, etc.) est une décision différée, pas un oubli.

Agents (5)

Q16. Combien d'agents métiers sont réellement actifs ?

19, enregistrés dans `agents/manifest.js`. Un 20e (`SearchAgent`) existe mais n'est plus câblé.

Q17. Tous les agents appellent-ils le LLM ?

Non — 6 agents (`KpiAgent`, `BusinessAnalystAgent`, `RecommendationAgent`, `DataQualityAgent`, `AdvancedSearchAgent`, `RoleCapabilityAuditAgent`) sont 100% déterministes et n'invoquent jamais le modèle.

Q18. Quel agent a le périmètre de Tools le plus large ?

`AutomationAgent` — aucune liste blanche (`ALLOWED_TOOL_NAMES=null`), compensé par une restriction de rôle plus stricte (administrer uniquement).

Q19. Comment un agent est-il sélectionné pour une intention donnée ?

`AgentRouter.select(intent)` délègue à `AgentRegistry.findHandler()`, qui retourne le premier agent dont `canHandle(intent)` est vrai.

Q20. Un agent peut-il utiliser à la fois le RAG et le Tool Calling ?

Un agent choisit l'un OU l'autre par tour — le RAG n'intervient qu'en repli si aucun Tool n'est jugé pertinent ; en pratique, seuls 2 agents documentaires l'utilisent réellement.

MCP (4)

Q21. MCP est-il un vrai serveur séparé en production ?

Non — le protocole JSON-RPC 2.0 est réel et conforme, mais le serveur MCP est instancié en mémoire dans le même process que l'application, jamais comme un second processus en production.

Q22. Combien de Tools MCP existent réellement ?

76, vérifié par exécution du code — les chiffres cités dans certains commentaires internes (69/73/74) sont obsolètes.

Q23. Un SDK MCP officiel est-il utilisé ?

Non — le protocole est implémenté à la main, le README du module le justifie par l'incompatibilité ESM-only du SDK officiel avec l'architecture CommonJS du projet.

Q24. Quel est l'ordre de vérification lors d'un appel de Tool ?

Validation de forme → Permission → Rate limiting → Validation métier (Joi) → Exécution → Audit — permission vérifiée avant même la validation de l'input métier.

Tool Calling (4)

Q25. Comment le modèle décide-t-il d'appeler un outil ?

Il produit un JSON strict à deux formes possibles (`call_tool` ou `final_answer`), interprété par `ToolCallingChain` — pas l'API de function-calling propriétaire d'un vendeur.

Q26. Que se passe-t-il si le modèle invente un nom d'outil inexistant ?

`UnknownToolError` est levée — l'appel n'atteint jamais le MCP.

Q27. Combien de tours d'outils maximum sont autorisés ?

5 (`maxToolCalls`) — au-delà, `ToolCallLimitExceededError` .

Q28. Comment le système empêche-t-il une hallucination de réussite d'outil ?

`requireToolUse` rejette tout `final_answer` si `steps.length===0` , fondé uniquement sur l'exécution réelle, jamais le texte du modèle.

Prompt Engineering (4)

Q29. Combien de niveaux de prompt système existent ?

Deux : des principes génériques + ton par rôle (5 rôles), et un prompt métier propre à chaque agent (20 agents en possèdent un) — non fusionnés.

Q30. Le prompt système peut-il empêcher une fuite de données ?

Non, seul — la garantie vient de mécanismes en code (RBAC, SourceGuard), le code le documente explicitement pour `CrmAssistantAgent`.

Q31. Comment le contexte RAG est-il injecté dans le prompt ?

Via `RagContextBuilder.buildContext()` , qui assemble citations et historique en une chaîne unique, scannée par le `PromptInjectionGuard` avant insertion.

Q32. Existe-t-il des instructions de sécurité explicites dans les prompts ?

Oui — par exemple, `PorPromeshAgent` interdit explicitement toute balise HTML dans la réponse, et plusieurs agents interdisent explicitement l'invention de valeurs absentes des résultats d'outils.

RBAC (5)

Q33. Combien de rôles existent réellement dans ce CRM ?

7, vérifiés dans l'ENUM Sequelize : user, commercial, accueil, admin, superadmin, superadmin2, responsable_logistique_achat.

Q34. Le rôle « accueil » a-t-il accès au Copilot IA ?

Non — explicitement exclu par `AI_COPILOT_ROLES_ALLOWED`, confirmé par test dédié.

Q35. Qu'est-ce que la fonction `isAuthorized()` ?

L'unique implémentation RBAC du kernel IA — un modèle hybride « rôle OU email », partagé sans duplication entre les Agents et le MCP.

Q36. `AgentRegistry` effectue-t-elle une vérification de permission ?

Non — c'est de la pure plomberie d'enregistrement/recherche ; le contrôle RBAC a lieu dans `BaseAgent.execute()`.

Q37. Comment `scopeInputForActingUser` protège-t-elle contre une fuite ?

Elle supprime les champs de portée (`ownerId` / `userId` / `mine`) fournis par le LLM pour tout rôle non admin-tier, et force le périmètre à l'utilisateur courant, quelle que soit la valeur reçue.

JWT (3)

Q38. D'où provient le rôle utilisé pour toutes les décisions RBAC ?

Exclusivement du JWT vérifié par `authRequired`, jamais du corps du message ni d'une décision du LLM.

Q39. Qu'est-ce que `actingUser` ?

L'objet normalisé `{id,email,role}`, traduit depuis le payload JWT `{sub,email,role}`, seul support transmis à l'orchestrateur IA.

Q40. Existe-t-il une double vérification d'accès HTTP au Copilot ?

Oui — `moduleAccessGuard` décode le JWT indépendamment avant `authRequired`, en plus de la porte applicative `canUseAiCopilot()`.

Prompt Injection (4)

Q41. Combien de règles de détection d'injection existent ?

5 patterns regex bilingues FR/EN, avec 3 modes (flag/sanitize/block).

Q42. Une attaque du type « `ownerId=...` » est-elle détectée par ce guard ?

Non, testé conceptuellement — aucune des 5 regex ne cible ce motif ; elle est neutralisée par d'autres mécanismes (Joi, scoping serveur).

Q43. Le guard protège-t-il aussi contre l'injection indirecte via le RAG ?

Oui — c'est même documenté comme le vecteur prioritaire : chaque extrait de citation est scanné avant insertion dans le prompt.

Q44. Que se passe-t-il en mode « block » face à une injection détectée ?

Une exception est levée, le texte n'atteint jamais le prompt du tout.

PendingAction (4)

Q45. Quels agents utilisent le mécanisme PendingAction ?

Trois : CommercialAgent, MeetingAgent, AutomationAgent.

Q46. Une double confirmation peut-elle exécuter une action deux fois ?

Non — `consume()` est appelé de façon atomique avant l'exécution ; une seconde confirmation ne retrouve rien à confirmer, vérifié par test.

Q47. Le client Flutter peut-il falsifier le contenu d'une action en attente ?

Non — Flutter ne renvoie jamais l'objet PendingAction, seulement les mots « confirme »/« annule » ; le serveur relit toujours son propre état stocké.

Q48. Quelle est la durée de vie d'une action en attente ?

15 minutes (TTL), au-delà de quoi elle est traitée comme expirée et purgée.

PostgreSQL (3)

Q49. Le module IA a-t-il ses propres tables PostgreSQL ?

Non, confirmé par recherche exhaustive — conversations, PendingActions et audit log sont tous en mémoire, volatiles.

Q50. Comment l'ownership est-il vérifié en base pour une relance commerciale ?

Comparaison explicite `contact.createdBy == actingUser.id` (ou rôle admin-tier) avant tout INSERT.

Q51. Une migration a-t-elle été nécessaire pour le Copilot ?

Oui — l'ajout de la valeur `copilot_meeting` à l'ENUM `entityType` de la table `calendar_event_syncs`.

Flutter (3)

Q52. Comment le Copilot est-il intégré dans la navigation de l'application ?

Une seule fois, au niveau du `ShellRoute / AppShell` — visible sur tout écran authentifié, sans modification des autres écrans.

Q53. Quel pattern de gestion d'état est utilisé côté Flutter ?

GetX, avec deux controllers séparés : `CopilotController` (état métier) et `CopilotUiController` (état visuel).

Q54. Comment le JWT est-il transmis aux requêtes du Copilot ?

Via l'intercepteur Dio partagé de toute l'application (`ApiClient`), jamais un second client HTTP dédié au Copilot.

Google Calendar / Performance / Tests / Architecture (3)

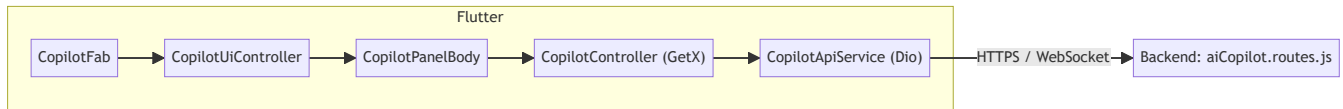
Q55. Que se passe-t-il si la création Google Calendar échoue après un Task CRM réussi ?

Le Task CRM reste en place (jamais annulé rétroactivement), l'échec est reflété honnêtement (`synced:false`), jamais présenté comme un succès.

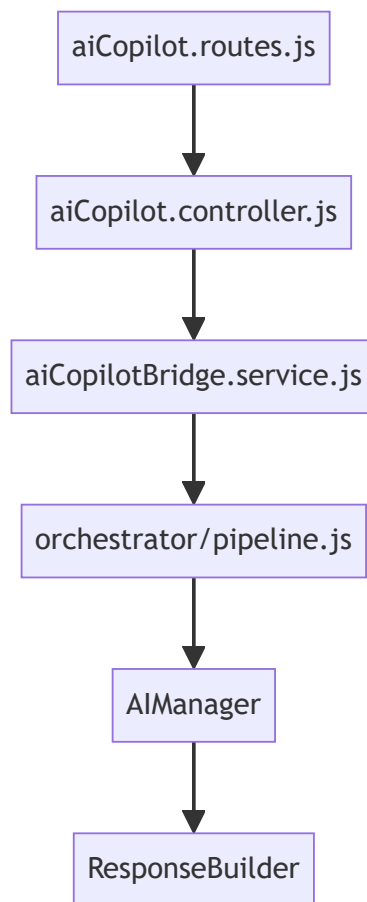
24. Diagrammes

Les diagrammes d'architecture globale, RAG, Tool Calling, PendingAction, Meeting et Google Calendar figurent respectivement en Figures des sections 3, 10, 11, 14 et 16. Les diagrammes complémentaires ci-dessous couvrent le Frontend, le Backend, le Multi-Agent, le LLM, le MCP, le RBAC et la Relance Commerciale.

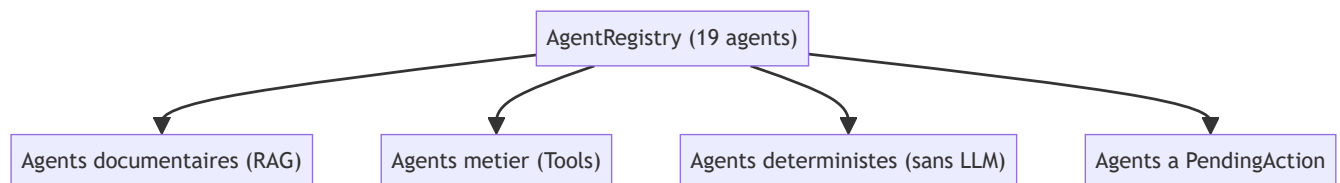
Architecture Frontend



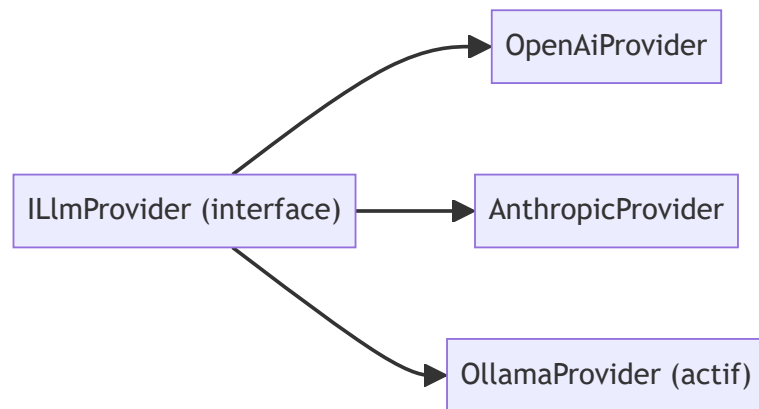
Architecture Backend



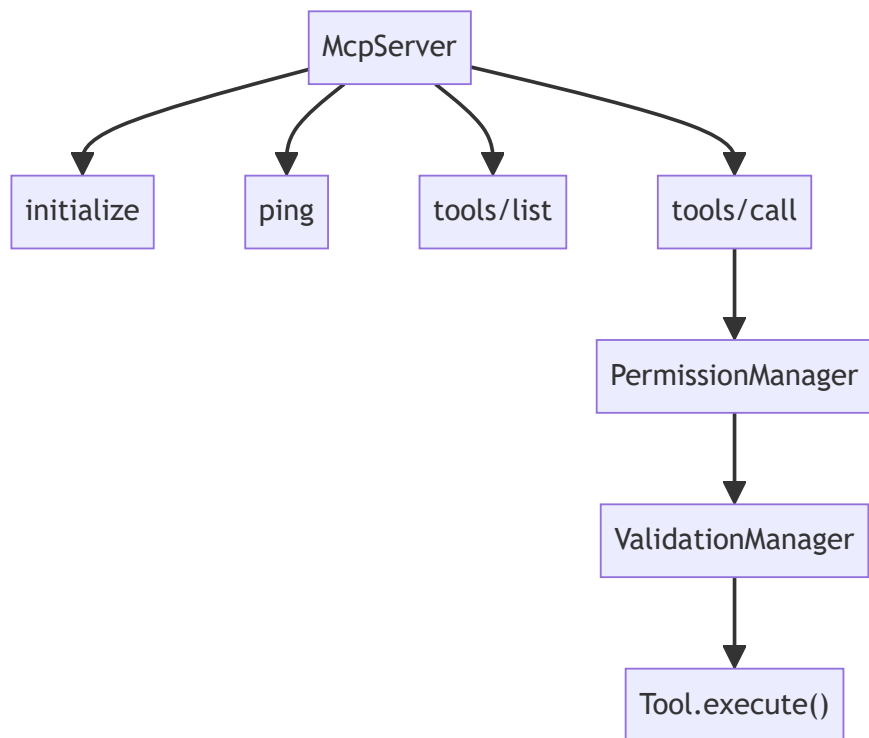
Multi-Agent



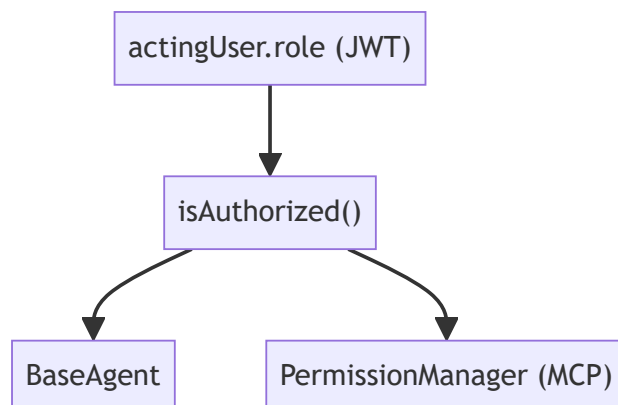
LLM (abstraction Provider)



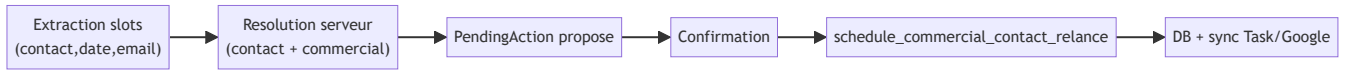
MCP



RBAC



Commercial Relance



25. Conclusion

Le Copilot IA du projet PROBAR constitue une réponse concrète et vérifiable à la problématique de navigation multi-module d'un CRM : il substitue à la traduction manuelle d'une intention métier en séquence d'écrans une interaction en langage naturel, sans jamais déléguer au modèle de langage la responsabilité de décider seul ce qu'un utilisateur est autorisé à voir ou à modifier. Les trois audits de code menés — architecture globale, sous-système LLM/RAG/Providers, et sécurité/RBAC — convergent vers un même constat : l'identité, le rôle et la portée des données d'un utilisateur ne dépendent jamais du texte échangé avec le LLM, mais exclusivement du JWT vérifié côté serveur et de vérifications mécaniques indépendantes (RBAC, validation Joi, comparaison d'ownership en base).

Cette rigueur architecturale coexiste avec des choix techniques volontairement simples pour les sous-systèmes où la complexité n'était pas justifiée à ce stade — un RAG par hachage en mémoire plutôt qu'une infrastructure vectorielle complète, un LLM local plutôt qu'une API payante — des choix documentés comme provisoires dans le code lui-même, et pour lesquels l'architecture en ports/adaptateurs anticipe déjà le chemin d'évolution. Les incidents réels rencontrés au cours du développement (hallucination de tool-calling sur POR PROMESH et sur les projets, fuite de portée sur les listes de projets) et leurs corrections vérifiées (`requireToolUse` , `scopeInputForActingUser`) illustrent une démarche d'ingénierie itérative fondée sur l'observation de défaillances réelles plutôt que sur une conception théorique a priori — une donnée méthodologique à part entière pour un mémoire de fin d'études en ingénierie logicielle.

Éléments à vérifier avant dépôt

Liste exhaustive des points signalés comme non confirmés, incomplets ou à revalider dans ce document, à traiter avant intégration finale dans le mémoire.

1. Nom exact du service/modèle/table pour `create_industrial_record` — le Tool et l'agent (`IndustrialAgent`) sont confirmés, mais le service métier et le nom exact de la table Sequelize sous-jacents n'ont pas été revérifiés dans cet audit. *Cette information n'a pas été trouvée dans le code analysé.*

2. Fichier de test précis pour le correctif RBAC de `getPersonalKpiSummaryTool.js` — la correction elle-même est confirmée dans le code, mais la référence exacte du test qui la verrouille n'a pas été localisée avec certitude. *Cette information n'a pas été trouvée dans le code analysé.*

3. Fichier de test précis pour `_syncCalendars` (relance commerciale) — le mécanisme est confirmé lu directement dans `commercialContactRelance.service.js`, mais aucun fichier de test dédié n'a été identifié avec certitude dans cet audit.

4. Résultats numériques d'exécution des suites de tests — le chiffre « 156 suites, 4721 tests, 100% verts » provient d'un document interne daté d'une passe antérieure du projet, non ré-exécuté au 29 août 2026. À confirmer par une exécution réelle de `npm test` avant toute citation dans le mémoire final.

5. Justification métier du choix du modèle `qwen3:8b` — seule la justification d'usage (local, sans coût d'API) est documentée dans le code ; aucune comparaison qualitative avec d'autres modèles Ollama n'a été trouvée.

6. Configuration active au moment du dépôt — cette analyse reflète le contenu du fichier `.env` réel au 29 août 2026 (`AI_LLM_PROVIDER=ollama`). Si cette configuration a changé avant la soutenance, les sections 8, 20 et les réponses Q1/Q4/Q5 doivent être revérifiées.

7. Écart de comptage documentaire (Tools et intentions) — le code exécuté donne 76 Tools et 21 intentions ; plusieurs fichiers `.md` internes citent des chiffres différents (69/73/74 Tools, 18 intentions). Le chiffre du code exécuté fait foi, mais cet écart mérite d'être mentionné explicitement à l'oral si le jury interroge la documentation interne.

8. Absence de vérification d'ownership dans `assignOwner()` / `moveStage()` — confirmée par lecture directe et documentée comme un choix assumé dans le code lui-même, mais à recontextualiser avec précaution si présentée comme une « faille » plutôt qu'une limite connue et volontairement compensée par le rôle.

9. Contrainte Joi du champ `role` dans `set_user_role` — le schéma accepte une chaîne libre (`max(50)`), la validation stricte contre l'ENUM réel étant déléguée à `user.service.js`, non audité en détail dans ce rapport.

10. Étendue réelle du corpus RAG au moment du dépôt — les racines documentaires (`docs/`, `AI-Business-Copilot/`, un PDF d'architecture) peuvent avoir évolué depuis cette analyse ; à revérifier si des exemples de citations RAG précis sont cités dans le mémoire.